U-Net Model



From

Satellite Images to Segmentation

Presented by - Meghana ON

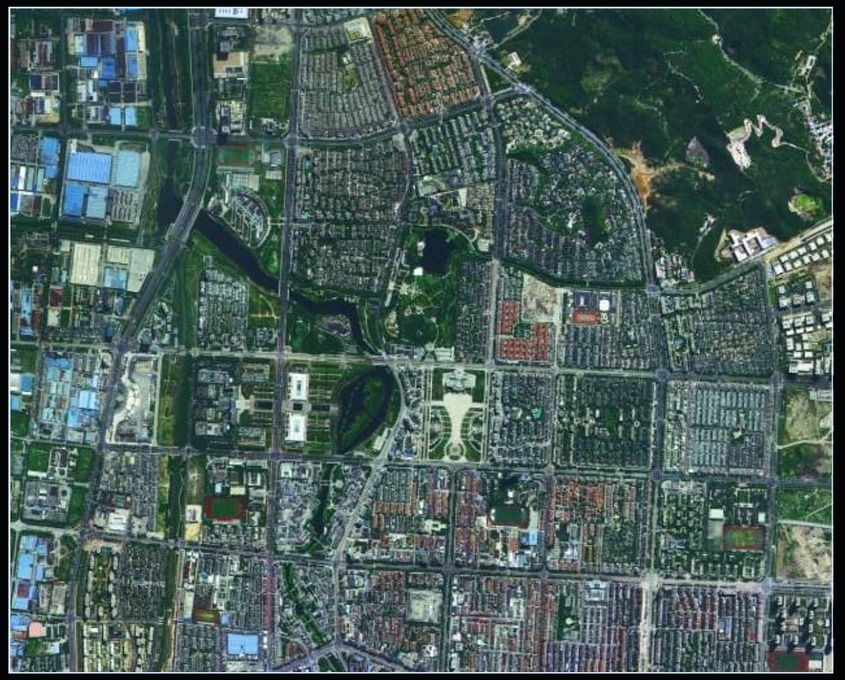
What Are We Trying to Do?

1) Satellite Image 🛰️



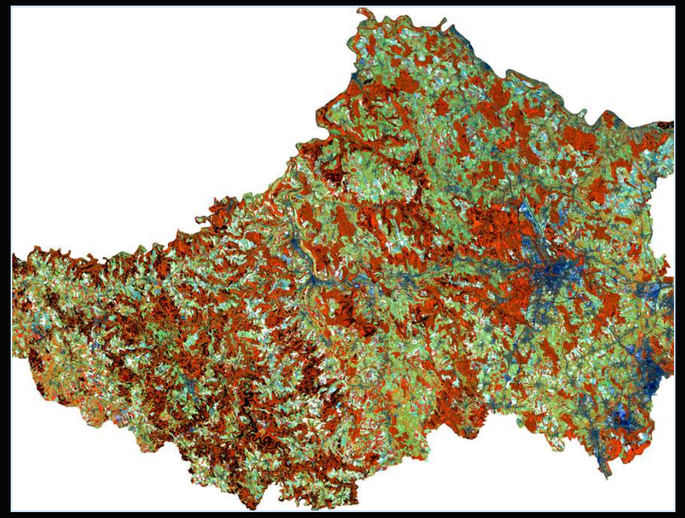
2) Identify Different Regions

🏠 Buildings
🛣️ Roads
🌳 Vegetation
🌊 Water



3) Segmentation Mask

Pixel-by-pixel output



What is Image Segmentation?

Breaking an image into meaningful regions — pixel by pixel

Satellite Image 🛰️



Look at each pixel



Breaking an image into meaningful regions — pixel by pixel

Segmentation Mask



Assign a region/class



So instead of just looking at the whole image, we want the computer to understand it pixel by pixel.

Why Do We Need U-Net?

Satellite Image

⬇️

Many pixels + different regions

⬇️

🏠 Buildings | 🛣️ Roads | 🌳 Vegetation | 🌊 Water

⬇️

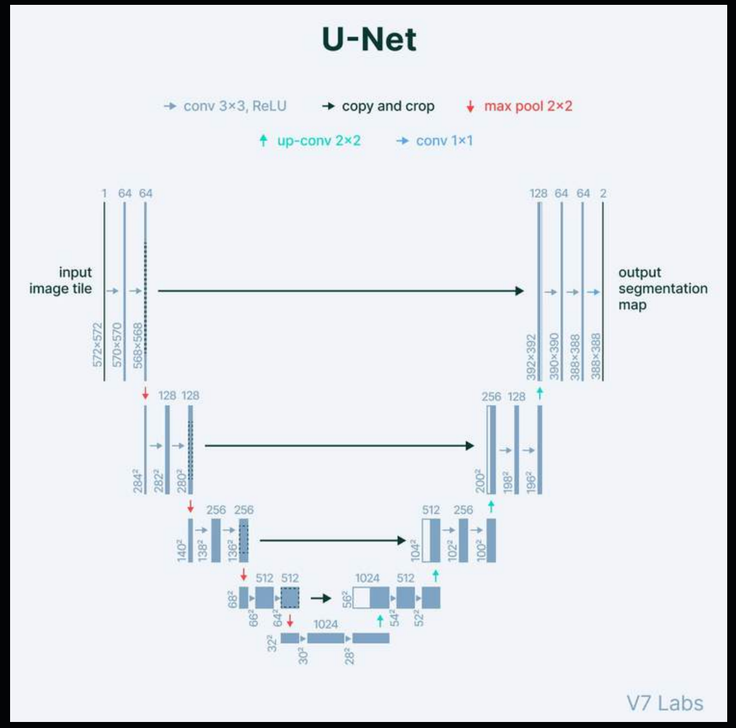
We need to identify every region pixel by pixel



U-Net learns which class each pixel belongs to.

What is U-Net?

U-Net is a CNN architecture designed for image segmentation.



🛰️ Satellite Image

🔵 Encoder - Understand the image

🟣 Bottleneck - Capture important features

🟢 Decoder - Recover the location/details

🎨 Segmentation Mask

Input → Understand → Reconstruct → Pixel-wise Mask

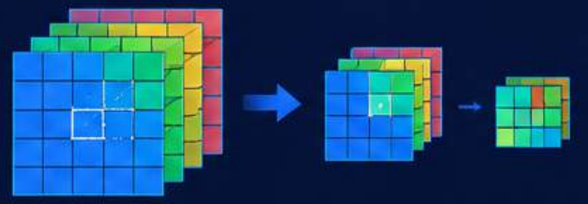
Encoder: Understanding the Image



Satellite Image

(Input)

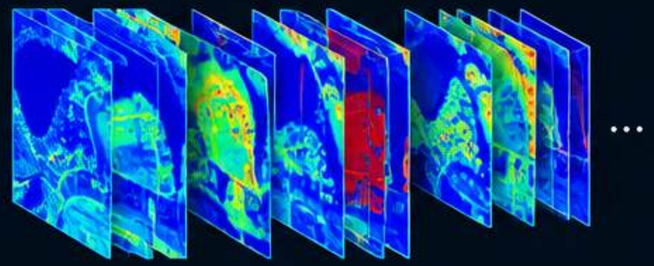
Convolution + Pooling



Convolution
(detects patterns)

Pooling
(reduces size)

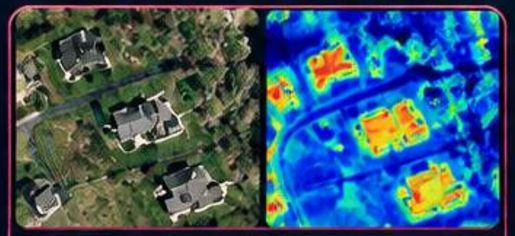
Feature Maps
(learned features)



Important Features

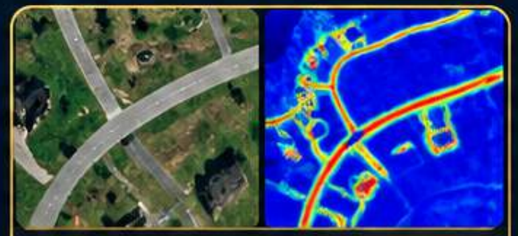
(Output from Encoder)

Examples of Learned Features



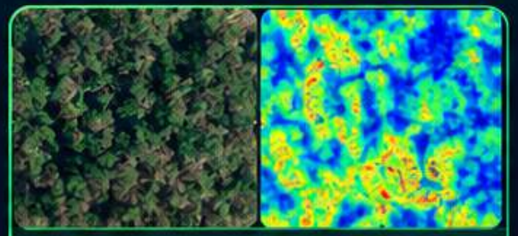
Building shapes

Detects edges and shapes of buildings.



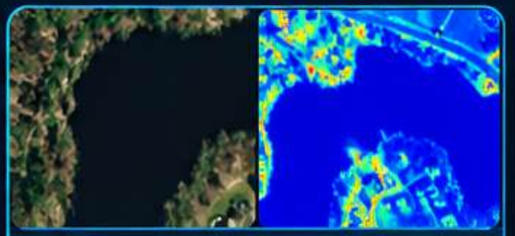
Road patterns

Identifies linear structures like roads.



Vegetation

Finds areas with high vegetation content.



Water regions

Detects water bodies like lakes and rivers.

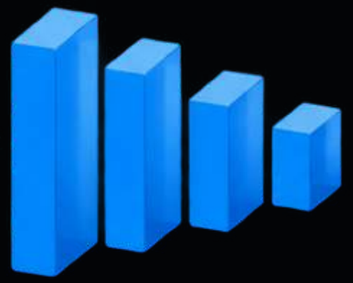
The encoder gradually **reduces the image size** while learning **important features**.

Step 2: Bottleneck – What Did U-Net Learn?

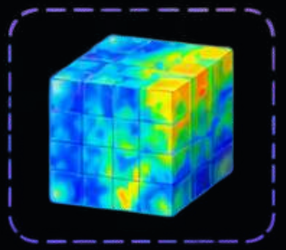
Satellite Image
(Input)



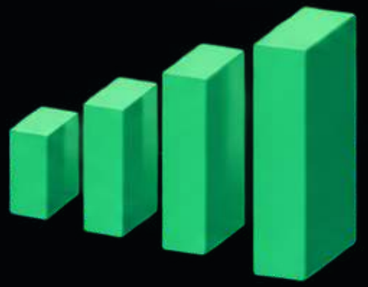
Encoder
(learns features)



Bottleneck
(compact representation)



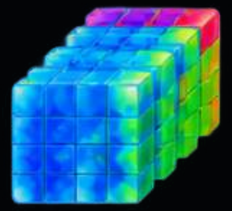
Decoder
(reconstructs details)



Segmentation Mask
(Output)

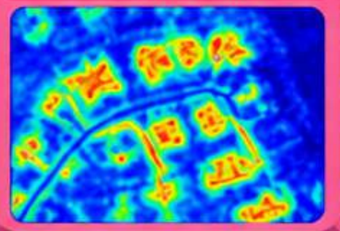


What the Bottleneck Learns



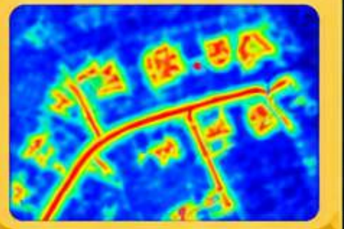
Compact Feature Maps
(smaller size, rich information)

Building shapes



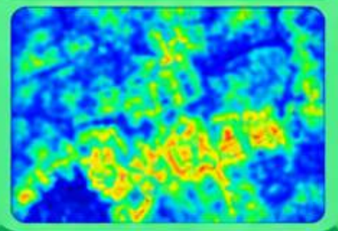
Learns edges and shapes of buildings

Road patterns



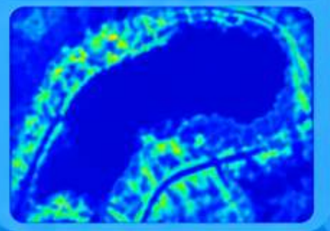
Identifies linear structures like roads

Vegetation patterns



Finds areas with high vegetation content

Water regions



Detects water bodies like lakes and rivers

The bottleneck contains a **compact representation** of the important information learned by the encoder.

Step 3: Decoder – Reconstruct the Details

The decoder takes the compact features from the bottleneck and gradually increases the image size, using skip connections to bring back the location and details.

Satellite Image
(Input)

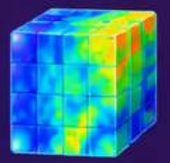


Encoder
(learns features)



Feature maps
(smaller size, more features)

Bottleneck
(compact representation)



Important features
(lowest resolution)

Decoder
(reconstructs details)



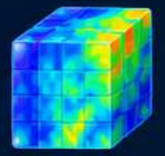
Feature maps
(larger size, detailed)

Segmentation Mask
(Output)

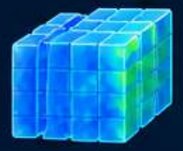


How the Decoder Works

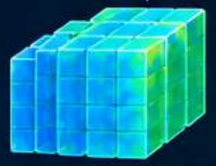
Skip Connections
(brings back fine details)



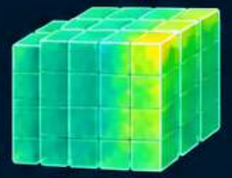
Bottleneck Output
(small, rich features)



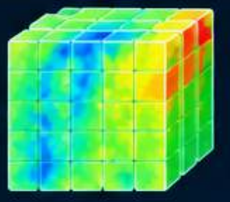
Upsampling
(increases size)



Upsampling
(increases size)



Upsampling
(increases size)



Final Features
(full size)



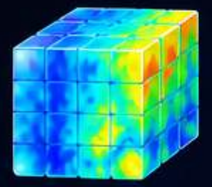
Segmentation Mask
(detailed & accurate)

The decoder gradually increases the image size and uses skip connections to recover the spatial details lost during encoding.

Step 4: Decoder – Reconstruct the Details

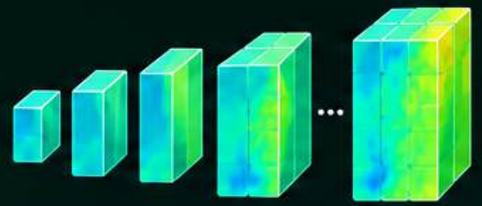
The decoder takes the compact features from the bottleneck and gradually increases the image size, using **skip connections** to bring back the location and details.

Bottleneck Output
(compact features)



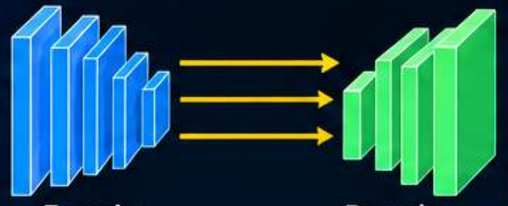
Small size, rich features

Decoder
(reconstructs details)



Upsampling (increases size)

Skip Connections
(brings back fine details)



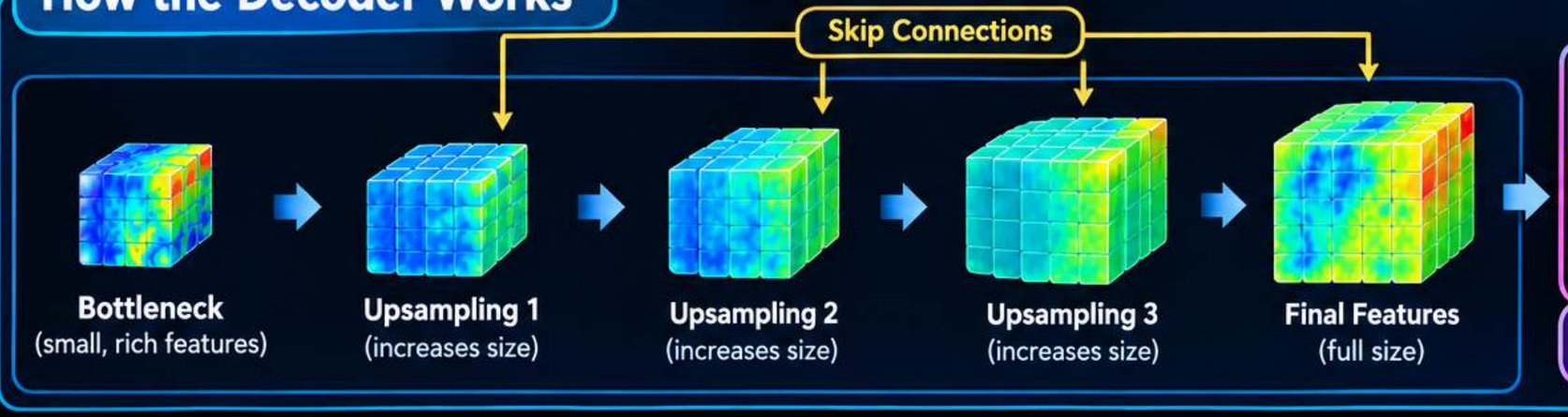
Encoder features

Decoder features

Segmentation Mask
(output)



How the Decoder Works



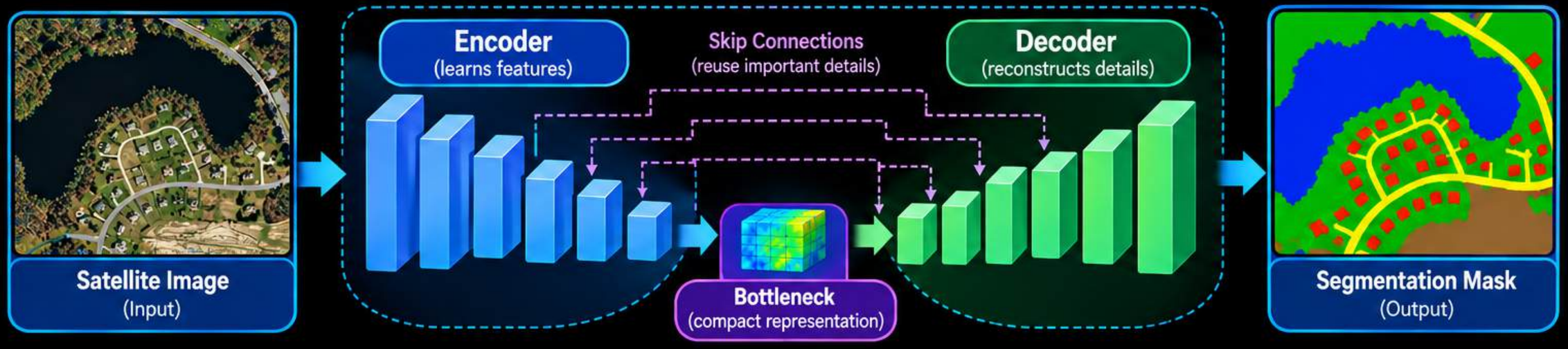


Segmentation Mask
(detailed & accurate)

The **decoder gradually** increases the image size and uses **skip connections** to recover the spatial details lost during encoding.

Step 5: The U-Net Architecture

U-Net is a **CNN** architecture designed for **image segmentation**.



What Each Part Does

Encoder

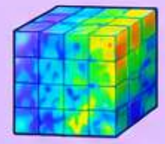
- Uses convolution + pooling
- Reduces image size
- Learns important features



(large → smaller)

Bottleneck

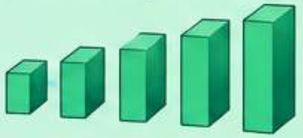
- Smallest feature map
- Stores compact information



(important features)

Decoder

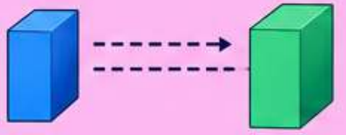
- Uses upsampling
- Reconstructs details
- Combines skip connections



(smaller → larger)

Skip Connections

- Brings back fine details
- Helps in better accuracy



(encoder → decoder)

Output

- Pixel-wise classification
- Each pixel gets a class
- Produces segmentation mask



(final result)

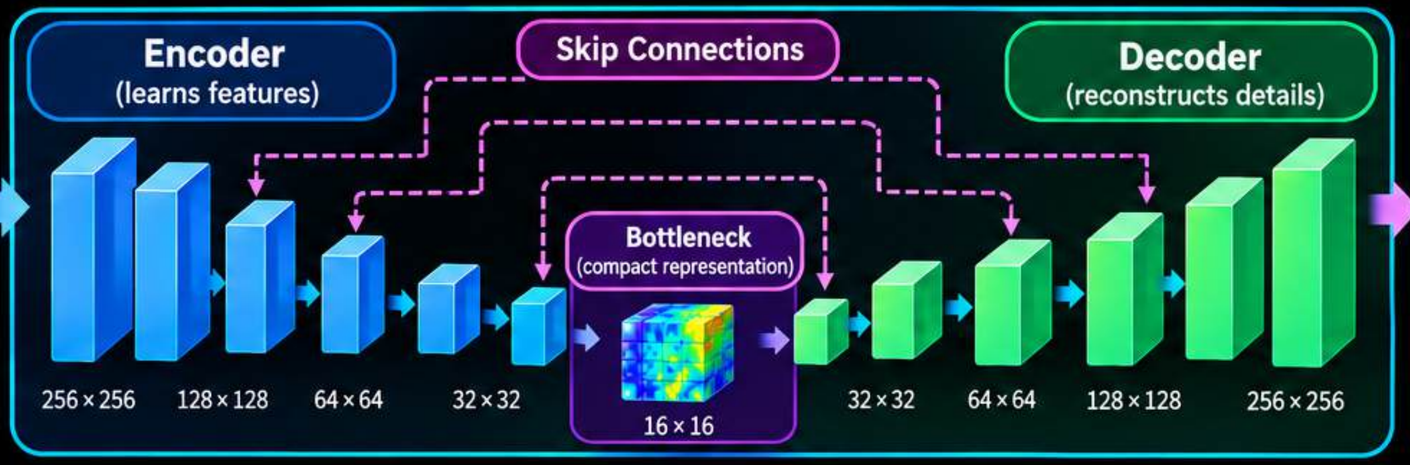
Input → Understand → Reconstruct → **Pixel-wise Mask**

Step 6: U-Net – Overall Architecture

U-Net combines the Encoder and Decoder with **skip connections** to get both the important features and the exact location details.



Input
(Satellite Image)





Output
(Segmentation Mask)

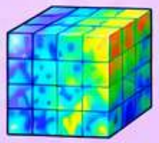
How It Works (Step by Step)

1 Encoder
- Convolution + Pooling
- Reduces image size
- Learns important features



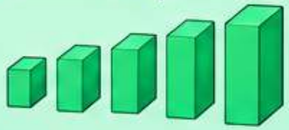
(large → smaller)

2 Bottleneck
- Most important features
- Compact representation



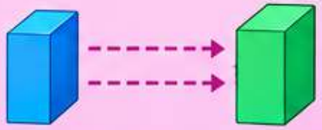
(small size, rich features)

3 Decoder
- Upsampling
- Reconstructs details
- Combines skip connections



(smaller → larger)

4 Skip Connections
- Brings back fine details
- Helps in better accuracy



(encoder → decoder)

5 Output
- Pixel-wise classification
- Each pixel gets a class
- Produces segmentation mask



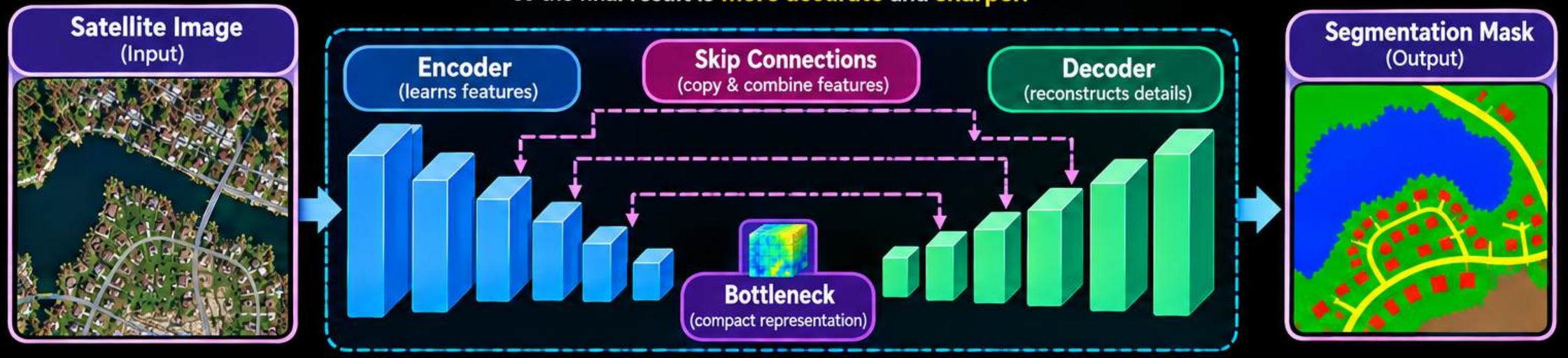
(final result)

U-Net = Encoder + Bottleneck + Decoder + Skip Connections

The perfect balance of feature learning and spatial detail!

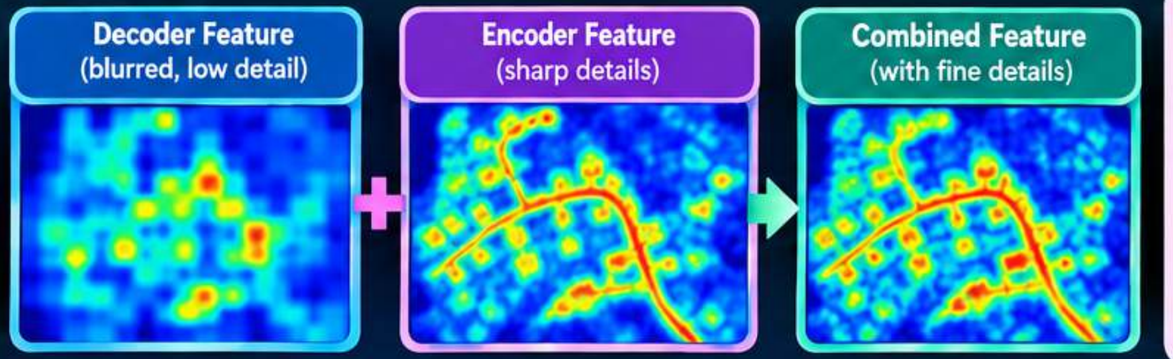
Step 7: Skip Connections – Bringing Back Fine Details

Skip connections copy important details from the encoder and combine them with the decoder, so the final result is **more accurate** and **sharper**.



What are Skip Connections?

- Passes high-resolution features from encoder to decoder.
- Helps recover fine details (like edges and small objects).
- Improves the accuracy of the final segmentation.



The Result

- Sharper boundaries
- More accurate shapes
- Better segmentation

Clearer details in the final output!

Skip connections = Better details + Higher accuracy = A clearer segmentation mask

Step 9: Final Segmentation Mask

The U-Net processes the satellite image and predicts a class for each pixel, resulting in a final segmentation mask.

Original Satellite Image



U-Net

Pixel-wise Classification



Final Segmentation Mask



- Red → Buildings
- Yellow → Roads
- Green → Vegetation
- Blue → Water

Each pixel is assigned to a class.

Step 10: Original Image vs Segmentation Mask

The original satellite image shows **what is present**,
while the segmentation mask shows **where each class is located**.

Original Satellite Image



Segmentation Mask



Class Legend

- → Buildings
- → Roads
- → Vegetation
- → Water

Key Differences

- Original image — real-world appearance
- Segmentation mask — class-based representation
- Shows exact location of each class
- Helps in better analysis and decision making

Original image → What the scene looks like.
Segmentation mask → What each pixel represents.

Step 11: Inside U-Net: From Image to Prediction

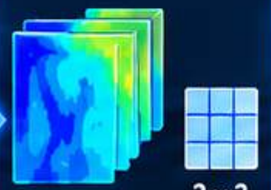
U-Net transforms a satellite image into a pixel-wise segmentation mask through a series of operations like convolution, pooling, upsampling and skip connections.

Input Image



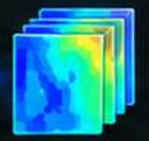
Satellite image
(H × W × 3)

Convolution
(Extract features)



(H × W × 32)
e.g. 32 filters

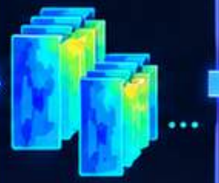
Pooling
(Reduce size)



MaxPooling 2 × 2

(H/2 × W/2 × 32)

Encoder
(Learn features)



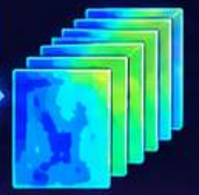
(smaller + deeper)

Bottleneck
(Compact (representation)



(H/16 × W/16 × 256)

Decoder
(Upsample + + Reconstruct)



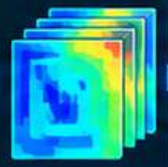
(H × W × 32)

Skip Connections
(Add fine details)



(from encoder to decoder)

Pixel-wise Prediction
(Class for each pixel)



(H × W × 4)
(4 classes)

Segmentation Mask



- = Buildings
- = Roads
- = Vegetation
- = Water

During Training



Input Image
(H × W × 3)



U-Net
(Prediction)



Prediction
(H × W × 4)

Loss Function
(e.g. Categorical Crossentropy + Dice Loss)

Backpropagation
(Calculate gradients)

Weight Update
(Improve model)

Better Prediction
(Lower loss, highee accuracy)

Key Concepts

1. Convolution – extracts features
2. Pooling – reduces size
3. Channels – number of feature maps
4. Upsampling – increases size
5. Skip Connections – recovers details
6. Loss – measures error
7. Backpropagation – updates weights
8. Softmax – class probabilities

Slide 12 Convolution + Kernel + Filters + Channels

Convolution uses small kernels (filters) to scan the image and extract important features.

Input Image



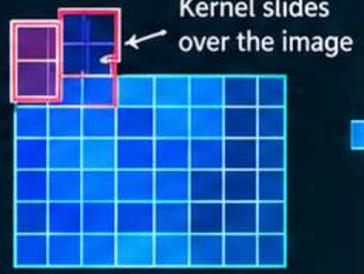
(H × W × 3)
RGB Image

Kernel (Filter)

-1	0	1
-1	0	1
-1	0	1

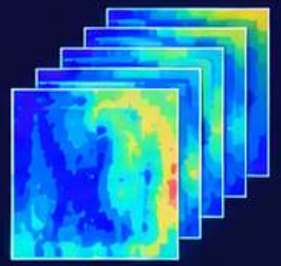
3 × 3 Kernel
(detects edges)

Convolution Operation



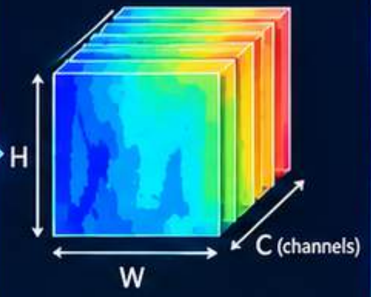
Produces a feature map

Feature Maps (Filters)



e.g. 32 filters
→ 32 feature maps

Channels



Output shape:
(H × W × C)

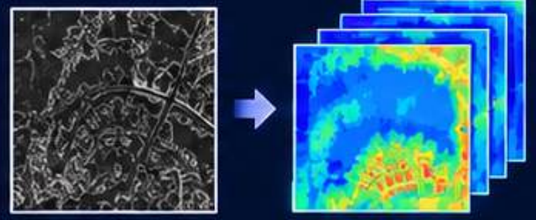
What It Does?

- Convolution extracts useful patterns.
- Kernel is the small matrix that scans the image.
- Filters learn different features.
- Channels represent the depth of the feature information.

Key Concepts

- **Convolution** → Extracts features
- **Kernel** → Small matrix (e.g. 3 × 3)
- **Filters** → Multiple kernels (e.g. 32)
- **Channels** → Feature map depth

Example Output



Input Image

Feature Maps
(from multiple filters)

In Code (Colab)

```
conv = nn.Conv2d(3, 32, kernel_size=3,
                 stride=1, padding=1)

x = conv(x)              # (B, 32, H, W)
x = F.relu(x)            # activation
x = F.max_pool2d(x, 2) # pooling
```

In our model, we used 3×3 kernels, 32 filters and ReLU activation.

Why We Use It?

- Helps the model learn important patterns in the image.
- Reduces noise and irrelevant details.
- Builds rich feature representations for better segmentation.

Convolution + Kernel + Filters + Channels → Extracts meaningful features from the image.

Slide 13 Stride – How Far the Kernel Moves

Stride tells the kernel **how many pixels** to move at each step while scanning the image.

What is Stride?

Stride is the step size of the kernel as it moves across the image.

Stride = 1 — Moves 1 pixel at a time

Stride = 2 — Moves 2 pixels at a time

Why is it Important?

- Controls the size of the feature map.
- Reduces computation and memory.
- Helps in learning better features at different levels.

Example: 3 × 3 Kernel on a 5 × 5 Image

Input Image (5 × 5)

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

*

3 × 3 Kernel

1	0	-1
1	0	-1
1	0	-1

Stride = 1



$(5 - 3 + 1) = 3$

Output size

Feature Map (3 × 3)

Stride = 2



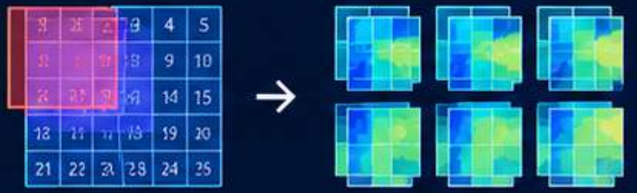
$(5 - 3) \div 2 + 1 = 2$

Output size

Feature Map (2 × 2)

How Does It Work?

Stride = 1 (overlapping windows)



More overlap (creates larger feature map)

Feature maps (3 × 3)

Stride = 2 (skipped pixels)



Less overlap (smaller feature map)

Feature maps (2 × 2)

Key Takeaway: Stride controls how the kernel moves, which affects the size of the feature map and the amount of computation.

Slide 14 Pooling & Activation (ReLU)

Pooling reduces the size of feature maps, and ReLU adds non-linearity to help the model learn complex patterns.

Pooling

Pooling takes the most important information from a region and reduces the spatial size of the feature map.

Example: Max Pooling (2 × 2)

1	3	2	1
5	6	4	2
3	2	1	0
1	2	3	4

Input Feature Map (4 × 4)

2 × 2 pooling (regions) →

6	4
3	4

Output Feature Map (2 × 2)

Why we use it?

- Reduces computational cost.
- Keeps important features.
- Helps prevent overfitting.

Activation (ReLU)

ReLU (Rectified Linear Unit) adds non-linearity to the model by setting negative values to zero.

Example

-1	2	-3
4	-5	6
-2	7	-8

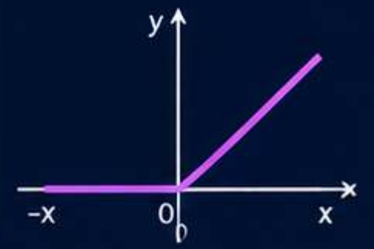
Input

→

0	2	0
4	0	6
0	7	0

After ReLU

ReLU(x) = max(0, x)



Why we use it?

- Introduces non-linearity (helps learn complex patterns).
- Keeps positive values, removes negative ones.
- Helps the model converge faster.

In short: **Pooling** = smaller & simpler | **ReLU** = non-linear & better learning

Slide 15 Upsampling & Decoder

Upsampling increases the **spatial size** of feature maps, and the **decoder** reconstructs the segmentation using the learned features and **skip connections**.

Upsampling

Upsampling increases the spatial size of the feature map (usually using Transposed Convolution or Interpolation).

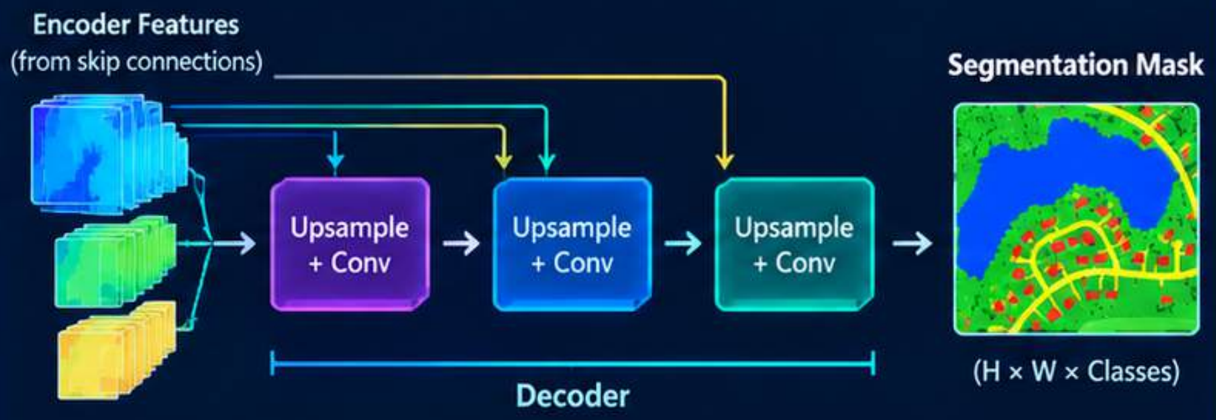


Why it's important?

- Restores spatial details lost during pooling.
- Helps the decoder reconstruct the full image size.
- Improves the accuracy of the segmentation mask.

Decoder

The decoder uses upsampled features and skip connections from the encoder to reconstruct the segmentation mask.



Why it's important?

- Combines high-level and low-level features.
- Recovers fine details and object boundaries.
- Produces a clear and accurate segmentation mask.

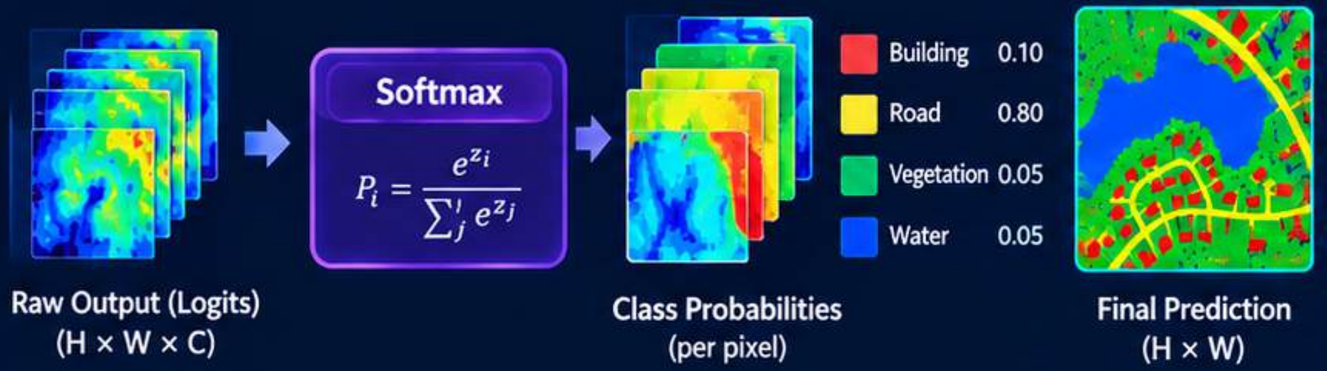
Upsampling (increases size) + **Decoder** (reconstructs image) → **Detailed Segmentation Mask** (with clear boundaries)

Slide 16 Softmax & Loss Function

Softmax converts the model output into **class probabilities**,
and the **loss function** measures the difference between the predicted and true masks.

Softmax

Softmax turns the raw output (logits) into probabilities **for each class.**



Raw Output (Logits)
(H × W × C)

Class Probabilities
(per pixel)

Final Prediction
(H × W)

Why it's important?

- Converts logits to probabilities (0 to 1).
- Helps the model choose the most likely class for each pixel.
- Works with multi-class segmentation.

Loss Function

The loss function measures how different the predicted mask is from the true mask. It helps the model learn during training.

Predicted Mask



True Mask



Loss (e.g., Cross Entropy)

$$L = -\frac{1}{N}\sum_{i=1}^{N} y_i \log(p_i)$$

Compares predicted and actual pixel values.

Why it's important?

- Guides the model to make better predictions.
- Penalizes incorrect predictions.
- Works along with backpropagation to update the weights.

Softmax = Class Probabilities | **Loss Function** = Guides Learning

Slide 17 Backpropagation & Weight Update

Backpropagation calculates how the **error** should affect the model's **weights**, and **weight update** adjusts the weights to **reduce the loss** during training.

Input Image



(H × W × C)

U-Net Model



(Forward Pass)

Prediction



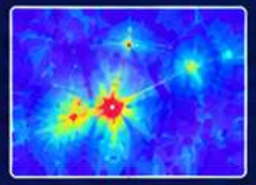
(H × W × Classes)

Loss

$$L = \text{Loss}(y, \hat{y})$$

(e.g., Cross Entropy)

Backpropagation



Calculate gradients ($\partial L / \partial w$)

Weight Update

$$w = w - \eta \frac{\partial L}{\partial w}$$

(Optimizer e.g., Adam)

Improved Model



(Better Prediction)

Backpropagation

- Calculates the **gradient** of the loss with respect to each weight.
- Shows how much each weight contributed to the error.
- Propagates the error from output to input (backwards).

Loss → Gradients → Backpropagation

Weight Update

- Uses the gradients to **adjust** the weights.
- Reduces the loss value.
- Helps the model learn better in the next iteration.

Old Weights → Update Rule → New Weights

Key Takeaway: **Backpropagation finds the error. Weight update fixes it.**
Together, they help U-Net learn better and give more accurate segmentation.

Slide 18 Model Evaluation

We evaluate the model using different **metrics** to check how well it predicts the segmentation mask.

Accuracy

Measures the percentage of correctly classified pixels.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$



Accuracy → Pixel correctness

IoU (Intersection over Union)

Measures the overlap between the predicted and actual regions.

$$\text{IoU} = \frac{TP}{TP + FP + FN}$$



IoU → Region overlap

Dice Score

Measures the similarity between the predicted and actual masks.

$$\text{Dice} = \frac{2 \times TP}{2 \times TP + FP + FN}$$



Dice → Mask similarity

In Short:

- **Accuracy** = Pixel correctness
- **IoU** = Region overlap
- **Dice** = Mask similarity

Slide 19 My U-Net Mini Project

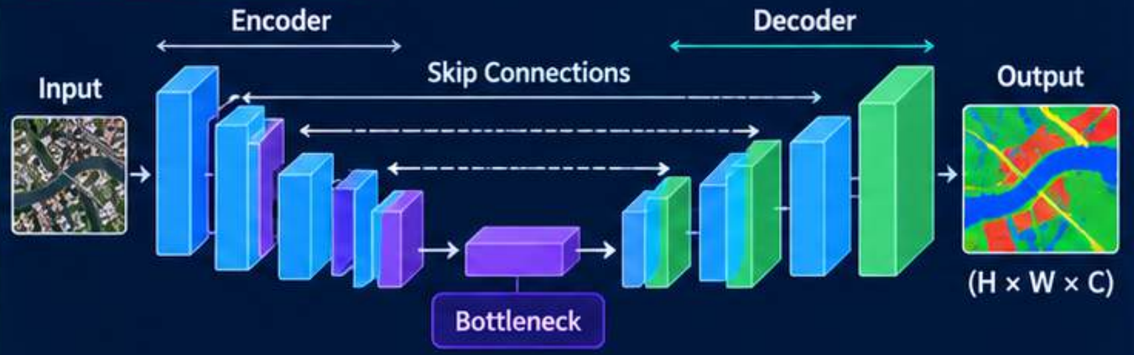
I implemented a small U-Net segmentation project in Google Colab using a satellite image.
The model learns from the image and predicts the different regions using the same concepts we discussed (convolution, pooling, decoder, loss and optimization).

1. Input Image (Satellite)



(H × W × 3)

2. U-Net Model



3. Predicted Segmentation Mask



(H × W × C)

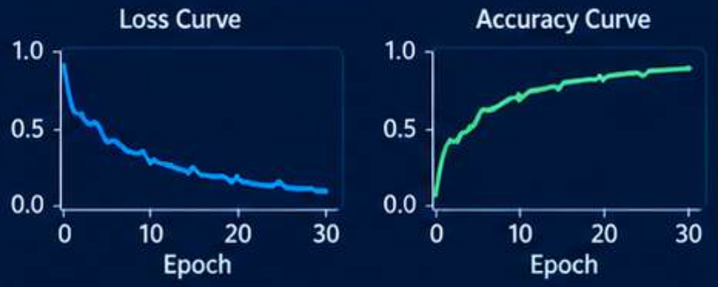
- Building
- Road
- Vegetation
- Water

4. Colab Code (Short View)

```
model = UNet(in_channels=3, out_channels=4)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

# Training loop
for epoch in range(epochs):
    outputs = model(images)
    loss = criterion(outputs, masks)
    loss.backward()
    optimizer.step()
```

5. Training Results



The loss decreases and accuracy increases as the model learns.

6. Evaluation Metrics

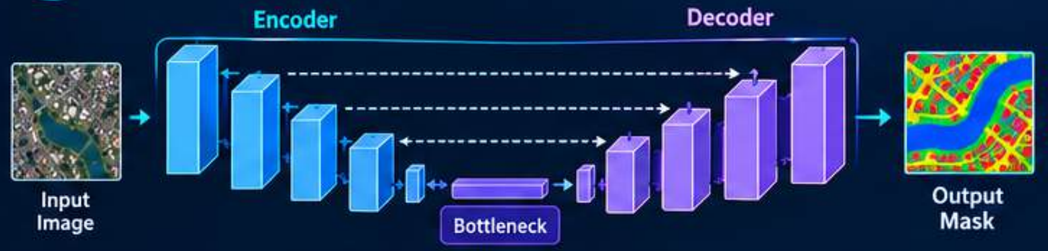
Accuracy	IoU	Dice Score
0.94	0.81	0.88
(pixel correctness)	(region overlap)	(mask similarity)

Key Takeaway

The U-Net model successfully segments the satellite image by learning from the data and predicting each region accurately.

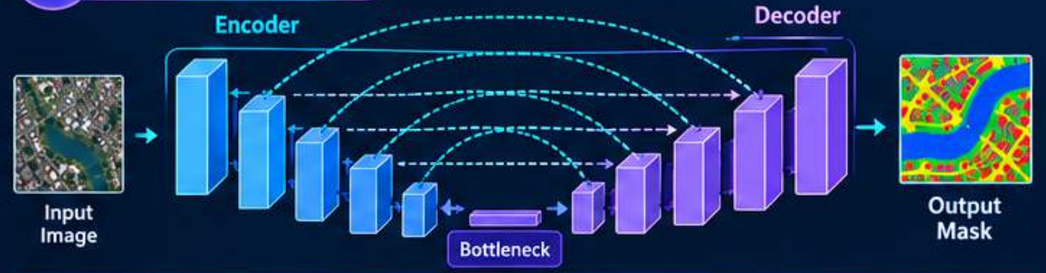
U-Net Family: Different Variants

1 Original U-Net



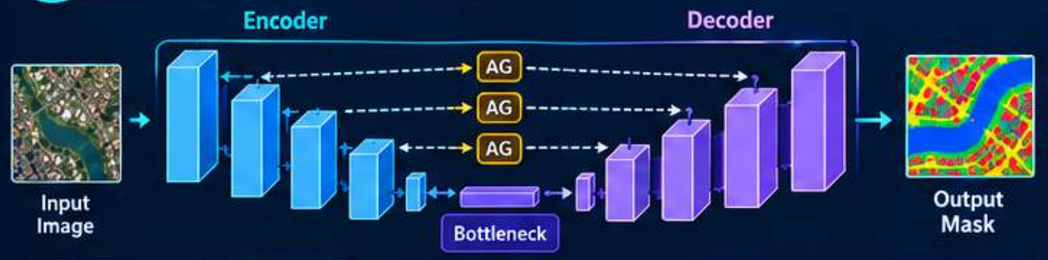


2 U-Net++



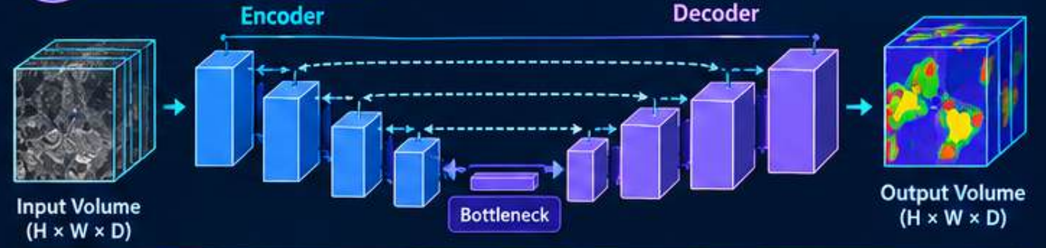


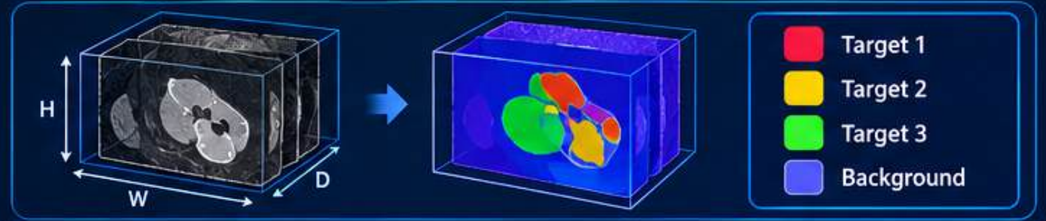
3 Attention U-Net





4 3D U-Net





Same U-Net idea → Different architectural improvements

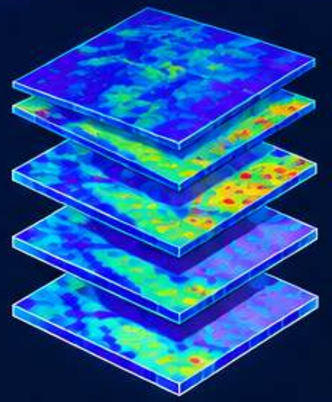
U-Net: How It Works

1 Satellite Image



Input image
(H × W × 3)

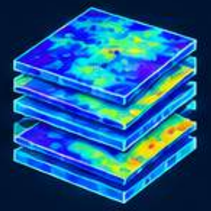
2 Feature Extraction



Convolution + ReLU
(Encoder)

Feature maps
(H × W × C)

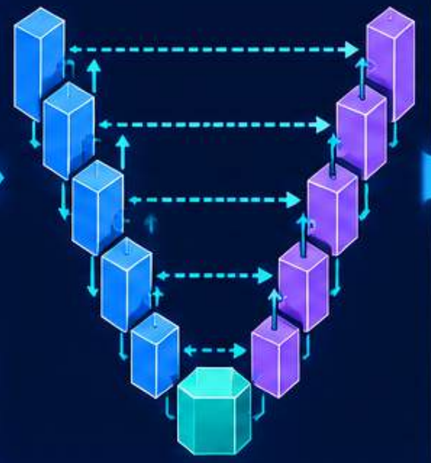
3 Bottleneck



Lowest resolution
(high-level features)

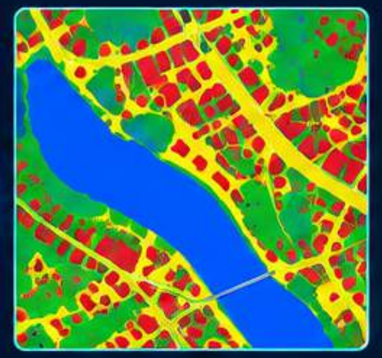
Feature maps
(H/16 × W/16 × C)

4 Decoder + Skip Connections



Upsampling + Concatenation
(Decoder)

5 Segmentation Mask



Output mask
(H × W)

- Buildings
- Roads
- Vegetation
- Water

U-Net learns what each pixel represents and where it is located.

U-Net++ : Segmentation Results

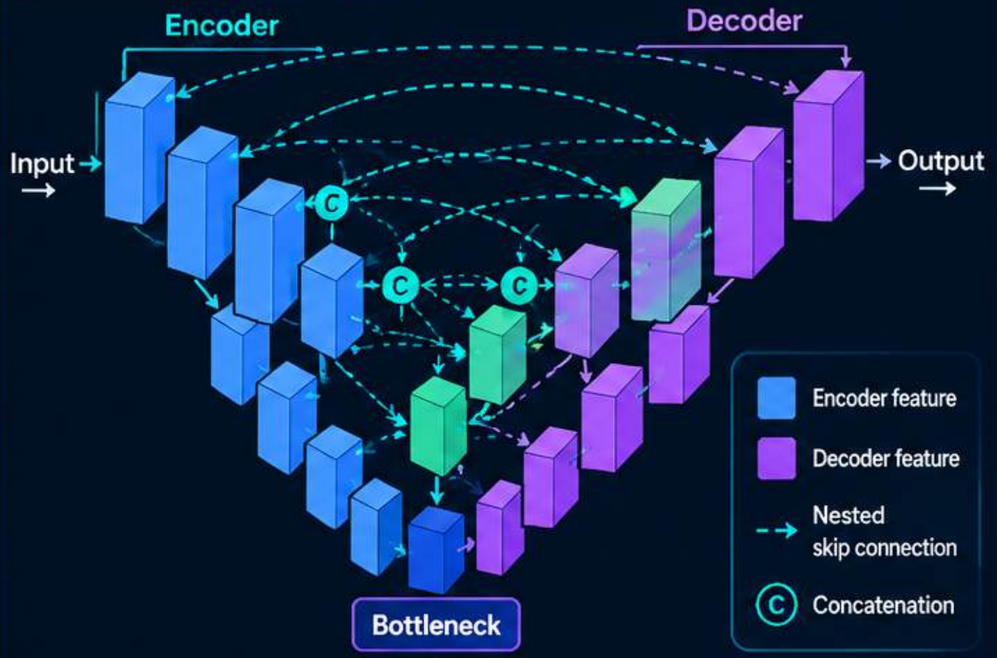
Nested skip connections • Better feature fusion • More accurate segmentation

1. Input Satellite Image



(H × W × 3)

2. U-Net++ Architecture



3. Segmentation Mask



(H × W)

Buildings
Roads
Vegetation
Water

U-Net++ Key Features



Nested Skip Connections
(multi-scale feature fusion)



Feature Concatenation
(more context + details)



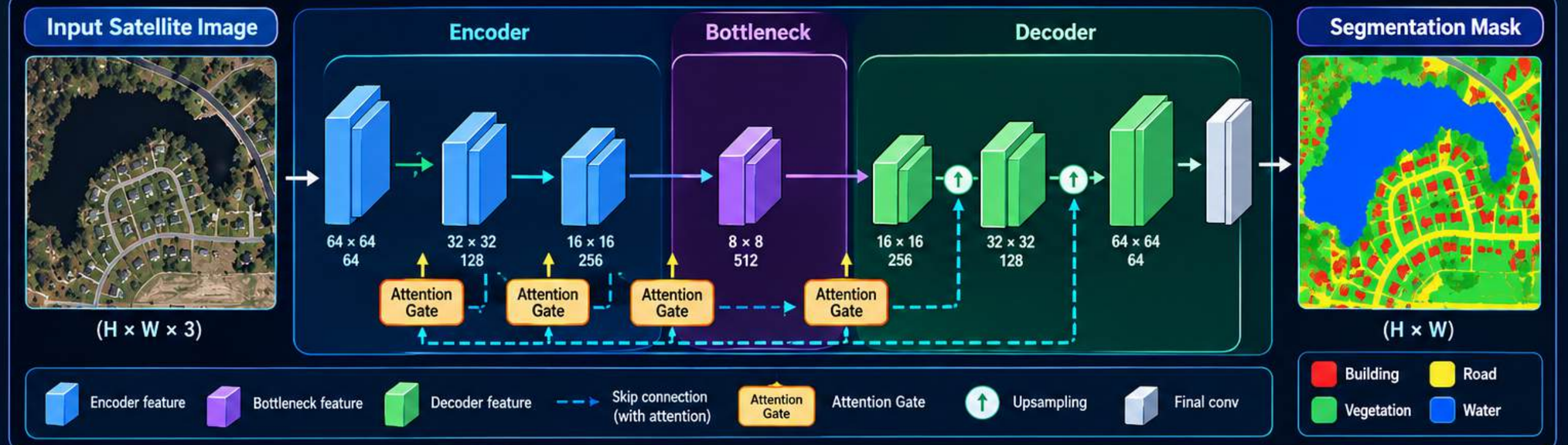
Better Boundary Accuracy
(sharp segmentation)



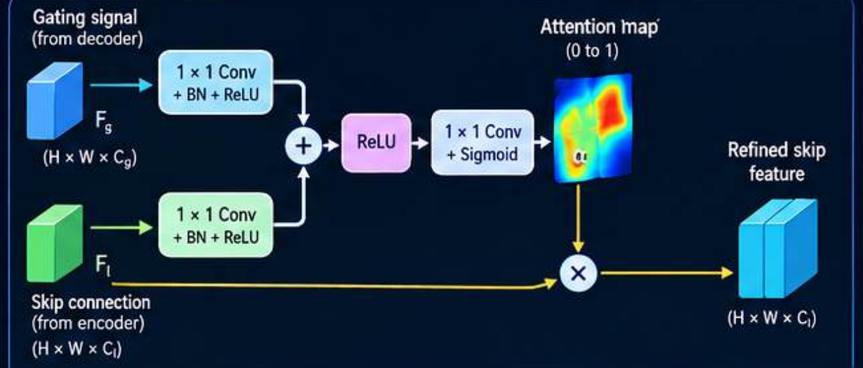
Higher Performance
(compared to original U-Net)

Attention U-Net

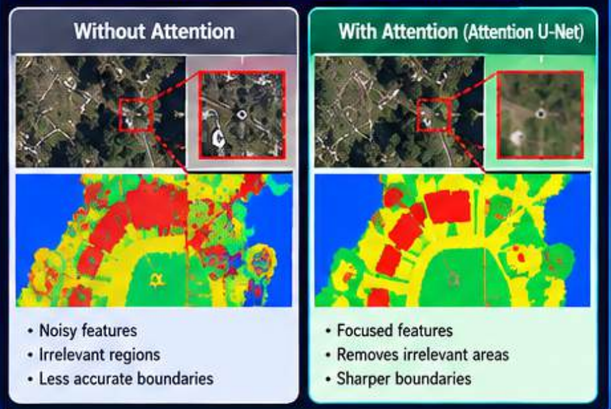
Focuses on important features • Reduces irrelevant information • Better segmentation accuracy



Attention Gate



Effect of Attention Gate on Skip Connection



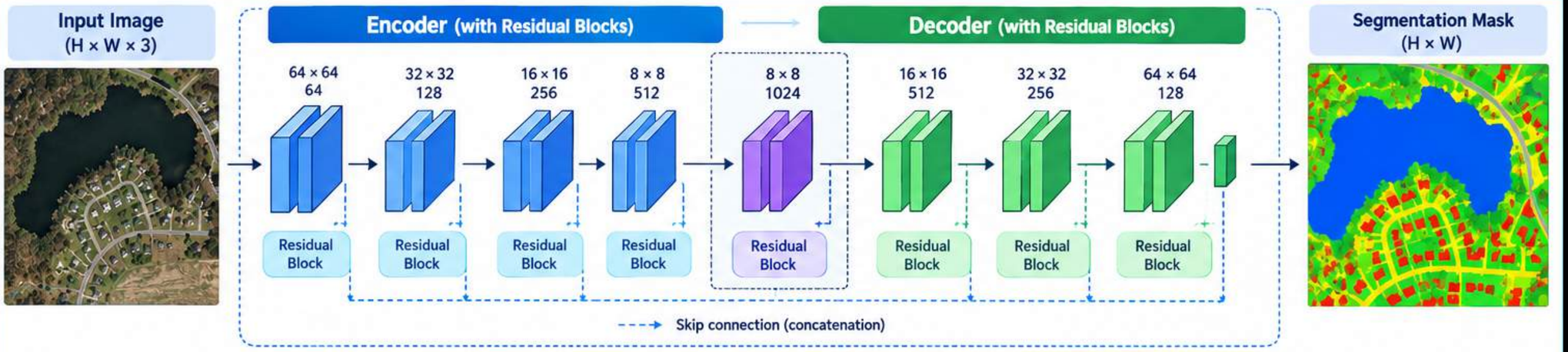
Key Benefits

- Focuses on important regions (attention mechanism)
- Suppresses irrelevant background (reduces noise)
- Improves segmentation accuracy (sharper boundaries)
- Works well for complex scenes (urban, rural, coastal, etc.)

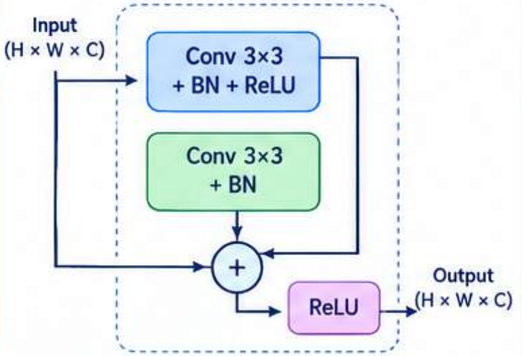
Residual U-Net (ResUNet)

U-Net + Residual Connections

Better gradient flow • Deeper network • Improved feature learning



Residual Block



The residual connection helps in:

- Better gradient flow
- Prevents vanishing gradient
- Allows deeper networks
- Improved feature learning

Key Components

Convolution
Extracts local features using filters (e.g., 3×3, 5×5).

Batch Normalization (BN)
Stabilizes training and speeds up convergence.

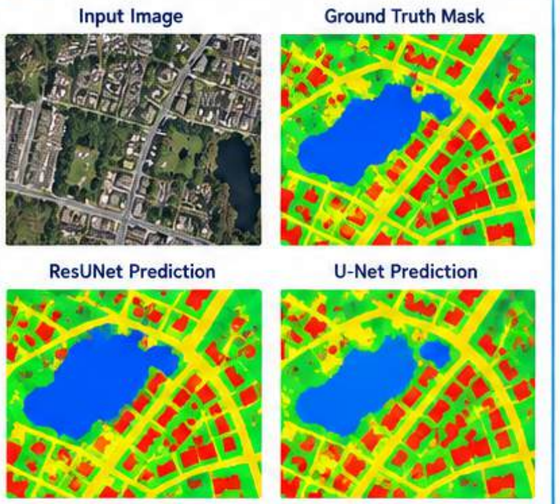
ReLU Activation
Introduces non-linearity (max(0, x)).

Residual Connection
Adds input to the output (x + F(x)).

Upsampling + Decoder
Reconstructs spatial resolution using transposed convolution / upsampling.

Skip Connections
Concatenate encoder features with decoder features.

Results on New Satellite Image



Why ResUNet?

Higher Accuracy
Residual learning helps the model learn better features.

Deeper Networks
Allows more layers without gradient vanishing.

Better Feature Reuse
Skip connections + residuals preserve important details.

Stable Training
Batch normalization + residual blocks improve convergence.

ResUNet = U-Net + Residual Blocks
= Stronger, Deeper, Smarter

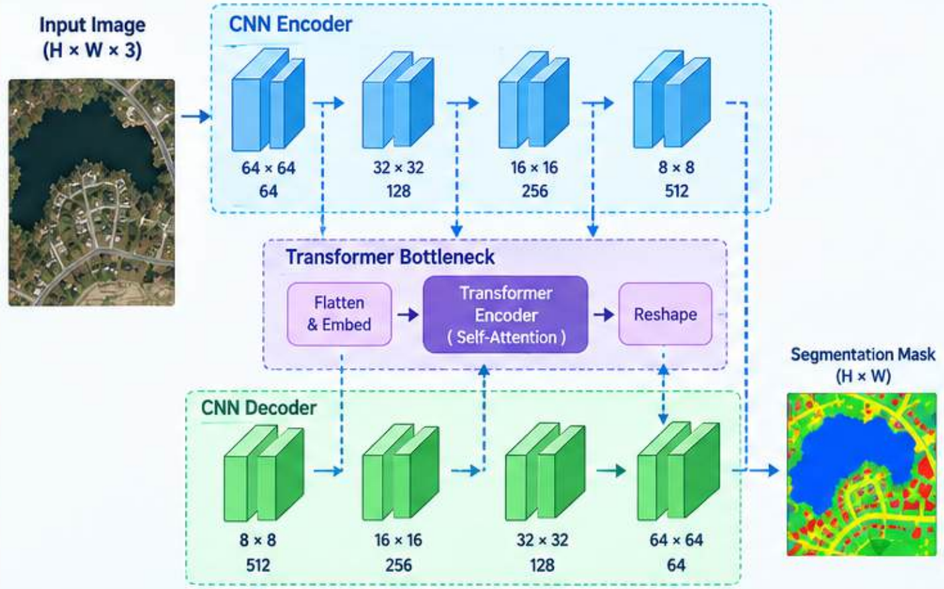
Transformer-Based U-Nets | (TransUNet / Swin-UNet)

Combining the strengths of CNNs and Transformers for better feature representation and segmentation

- Global context
- Better feature extraction
- Improved segmentation accuracy

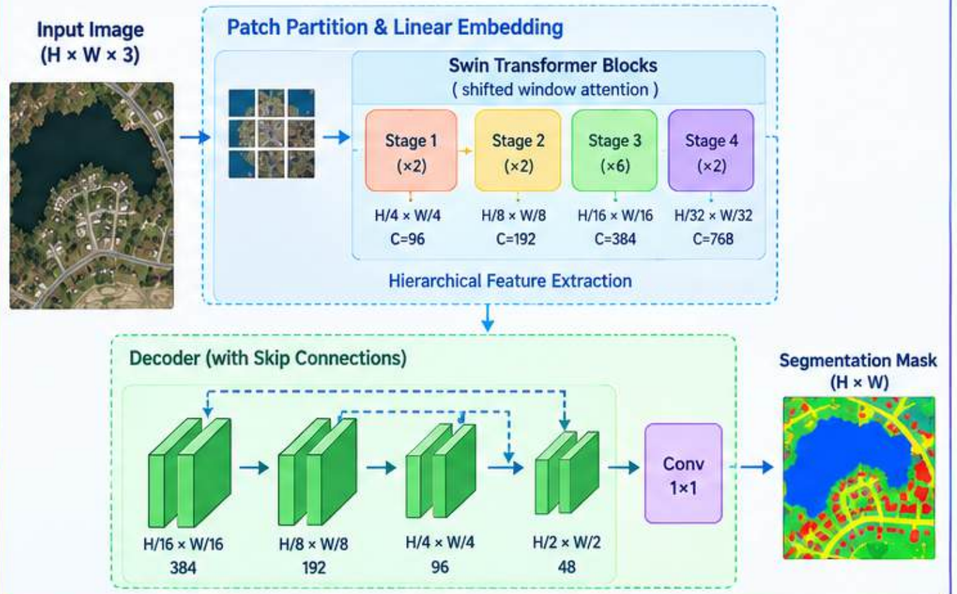
TransUNet

CNN Encoder + Transformer Bottleneck + CNN Decoder



Swin-UNet

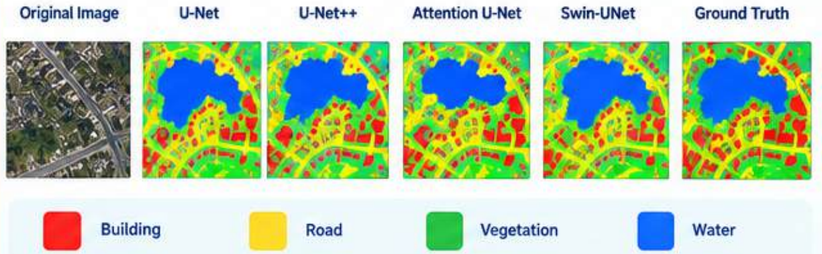
Hierarchical Swin Transformer + U-Net Architecture



Architecture Comparison

Feature	TransUNet	Swin-UNet
Transformer Type	Global Self-Attention	Shifted Window Attention
Feature Extraction	CNN + Transformer	Hierarchical Transformer
Attention Mechanism	Full Image (global)	Local windows (efficient)
Parameters (approx.)	~ 105M	~ 62M
Strength	Good global context	Better efficiency + multi-scale

Sample Segmentation Output (New Satellite Image)



Why Transformer-Based U-Nets?

- **Global context**
  Understands long-range dependencies in the image.
- **Better accuracy**
  Captures complex structures (e.g., buildings, roads, water).
- **Efficient**
  Swin's window attention reduces computational cost.
- **State-of-the-art**
  Achieves higher IoU / Dice in many remote sensing tasks.